Permutation Tests for 2 Samples - Examples in R

B.A. Hartlaub and A.S. Wagaman

The Wilcoxon Rank Sum test (as well as Fligner-Policello statistic and van der Waerden's test) is used to investigate the location problem in the 2 independent samples setting. Another nonparametric procedure that can be used is a permutation procedure. Permutation or other randomization-based procedures can be applied in many different situations, and are computationally intensive. With modern-day computing power, they are easy to implement and have gained popularity.

Permutation Tests

In a permutation test, you investigate how unusual your observed statistic is based on considering other arrangements, *permutations*, of the data, and with the statistic (or summary measure) of interest computed for each permutation. In the two independent samples setting, the idea is that under the null hypothesis (where there is no difference in the distributions) the observations in the Y sample are there simply by random chance. They just as easily could have been in the X sample. So, we will investigate possible permutations by considering all the observations as coming from the same distribution, and randomly assigning observations to Y (the same number of observations as Y originally had). The other observations go to X. Then, we compute our statistic for that permutation of the data. We repeat this process investigating many permutations, and see how extreme our observed statistic (for the actual data) is. If it is extreme, then our data provide an unusual result assuming the null hypothesis is true, and we would reject the null hypothesis.

This sounds pretty intense, but let's look at it through an example to see the process and how it works.

Example 1 - Test Prep Scores

In this example, we are comparing percentile test scores for students who enrolled in a test prep course taught using the traditional method versus those on a new method. All 7 students had similar test percentiles on a pre-test.

```
newmethod <- c(78, 88, 92, 93)
tradmethod <- c(73, 76, 81)

mean(newmethod) - mean(tradmethod)
```

```
## [1] 11.08333
```

```
alldata <- c(newmethod, tradmethod)
```

Our new method is considered the X sample and has 4 observations. Traditional is our Y sample and has 3 observations. Here, there are 7 total observations, and you could actually list every possible permutation. In other words, you could list all the different ways that you could assign 3 values to Y from the pool of 7 values. The number of possible permutations is:

```
choose(7, 3)
```

```
## [1] 35
```

Now, you could sit here and write out all 35 possible permutations. But that would be slightly tedious and you won't learn much from it. However, we can do this with the computer fairly easily. Once we list out all

the permutations, for each case, we record the difference in sample means. This will help to see how unusual the original difference in sample means (computed above as ~ 11.08) is, based on what could have happened with random assignment of observations to the groups.

```
# Insert code to get all permutations and
# record sample mean and put in a nice table

allperms <- combn(alldata, 3) # each column is a permutation
allperms <- data.frame(t(allperms))

# to help compute diff in means
total <- sum(alldata)

allperms2 <- mutate(allperms, ysum = X1 + X2 + X3,
                    xmean = (total-ysum)/4,
                    ymean = (X1 + X2 + X3)/3,
                    diff = xmean - ymean) %>%
  select(-ysum) %>%
  arrange(desc(diff)) %>%
  rename(Y1 = X1, Y2 = X2, Y3 = X3) # make variable names match

allperms2 %>% kable(booktabs = TRUE)
```

Y1	Y2	Y3	xmean	ymean	diff
78	73	76	88.50	75.66667	12.8333333
73	76	81	87.75	76.66667	11.0833333
78	73	81	87.25	77.33333	9.9166667
78	76	81	86.50	78.33333	8.1666667
88	73	76	86.00	79.00000	7.0000000
78	88	73	85.50	79.66667	5.8333333
92	73	76	85.00	80.33333	4.6666667
78	88	76	84.75	80.66667	4.0833333
88	73	81	84.75	80.66667	4.0833333
93	73	76	84.75	80.66667	4.0833333
78	92	73	84.50	81.00000	3.5000000
78	93	73	84.25	81.33333	2.9166667
88	76	81	84.00	81.66667	2.3333333
78	92	76	83.75	82.00000	1.7500000
92	73	81	83.75	82.00000	1.7500000
78	88	81	83.50	82.33333	1.1666667
78	93	76	83.50	82.33333	1.1666667
93	73	81	83.50	82.33333	1.1666667
92	76	81	83.00	83.00000	0.0000000
93	76	81	82.75	83.33333	-0.5833333
78	92	81	82.50	83.66667	-1.1666667
78	93	81	82.25	84.00000	-1.7500000
88	92	73	82.00	84.33333	-2.3333333
88	93	73	81.75	84.66667	-2.9166667
88	92	76	81.25	85.33333	-4.0833333
88	93	76	81.00	85.66667	-4.6666667
78	88	92	80.75	86.00000	-5.2500000
92	93	73	80.75	86.00000	-5.2500000
78	88	93	80.50	86.33333	-5.8333333
88	92	81	80.00	87.00000	-7.0000000

Y1	Y2	Y3	xmean	ymean	diff
92	93	76	80.00	87.00000	-7.0000000
88	93	81	79.75	87.33333	-7.5833333
78	92	93	79.50	87.66667	-8.1666667
92	93	81	78.75	88.66667	-9.9166667
88	92	93	77.00	91.00000	-14.0000000

For the permutation test, what we do then is consider the following question: "How unusual is 11.08 as a difference in means, when considering all possible permutations of the data?" The direction for "unusual" - upper-tail, lower-tail, or 2-sided depends on our question of interest. Here, we wanted to see if the new method was better than the traditional. Thus, we could obtain a p-value as the probability of obtaining 11.08 or higher as the difference in means for permuted samples.

Look at the listed permutations (for the Y data only), and the reported sample means and difference in means. Only one permutation has a larger difference, and then one permutation matches the data itself. This means that only two of 35 potential samples give differences in means as large or larger than what we observed in our data. So the empirical p-value is:

```
2/35
```

```
## [1] 0.05714286
```

Thus, our permutation based-p-value is 0.05714.

This is the basis for a permutation test in the two-sample setting. We will just permute the data to examine different possibilities for the breakdown into the two groups, under the null hypothesis assumption that the two distributions are the same. You get to choose the statistic you compare (the mean is common, but you could use the median too). These tests are nonparametric because no distributional assumptions are made about the underlying populations (other than they are the same under the null, and an assumption that random assignment would be reasonable).

Now, you may think that it's a little tedious to list out all possible permutations. Indeed, it is. In fact, for samples of size m and n, the number of possible permutations can be computed as:

```
m <- 10
n <- 15
possperm <- choose(m+n, m); possperm
```

```
## [1] 3268760
```

Note that with m=10, and n=15, there are more than 3 million possible permutations. How will we get around this? We won't try to list all the permutations. Instead, we use computers, and we randomly simulate possible permutations instead of listing them all. For example, we could generate 10000 possible permutations of the data set, instead of listing all 3 million permutations. As long as we randomly generate permutations, and get a fairly large number of them, the distribution of the statistic we observe based on the permutations should closely match the distribution we would obtain if we listed all the possible permutations.

Let's use R to do this! We will use a new package (you will need to install it once) called *exactRankTests*.

```
library(exactRankTests) #install once before it will load
# Note: exactRankTests is no longer under development
# In the future, may need to swap to coin package
# Alternative ways to do this exist in the coin package, with different structure
```

In this package, the *perm.test* function will do permutation tests for us using the mean. Let's try it for this data:

```
result <- perm.test(newmethod, tradmethod, alt = "greater")
result
```

```
##
##  2-sample Permutation Test
##
## data:  newmethod and tradmethod
## T = 351, p-value = 0.05714
## alternative hypothesis: true mu is greater than 0
```

In this example, the test gives us the exact result - it doesn't need to simulate permutations because there are only 35. How can we check that simulating would work? Let's write some code to let us examine the permutation density. (You are not expected to be able to code this, but do try to understand the density picture that results.)

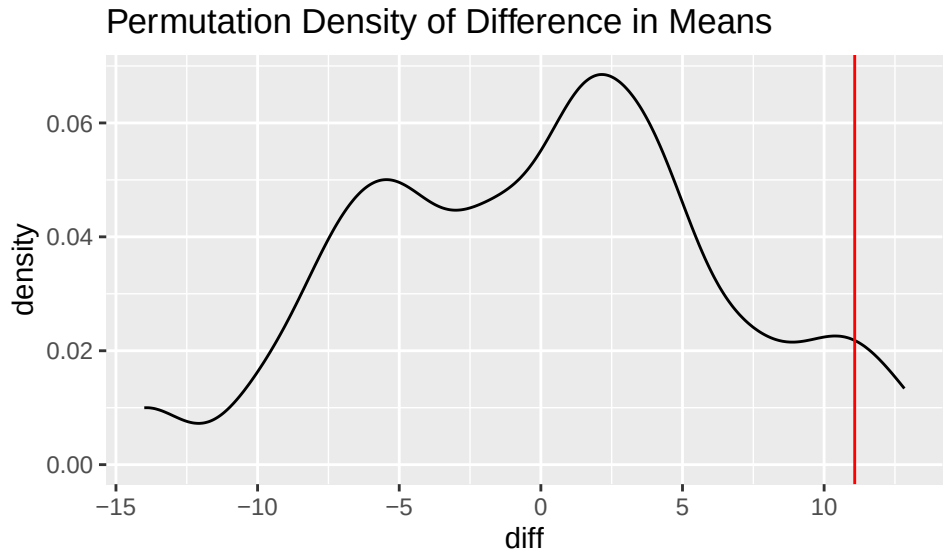
```
set.seed(225)
runs <- 1000
diff <- rep(0,runs)

sampledata <- c(newmethod, tradmethod)
total <- sum(sampledata)

for(i in 1:runs){
  sampley <- sample(sampledata,3) # randomly select observations for smaller sample
  sumy <- sum(sampley)
  diff[i] <- (total-sumy)/4-sumy/3 # creatively compute difference in means
}

df <- data.frame(diff)

ggplot(data = df, aes(x = diff)) +
  geom_density() +
  geom_vline(xintercept = 11.08, color = "red") +
  labs(title = "Permutation Density of Difference in Means")
```



These plots show the distribution of the differences in means for 10000 randomly generated permutations of

the data. We added a bar at 11.08 so we can see where our observed value was. We can also use pdata and qdata functions from mosaic to get p-values and critical values for CIs based on these randomly generated distributions of differences in means.

```
pdata(~ diff, 11.08, lower.tail = FALSE, data = df)
```

```
## [1] 0.065
```

```
qdata(~ diff, 0.95, data = df)
```

```
##      95%
## 11.08333
```

However, if you want a CI, you can turn that on as perm.test option, though you need to run the two-sided test for a CI (one-sided gives a confidence bound).

```
perm.test(newmethod, tradmethod, conf.int = TRUE)
```

```
## Warning in perm.test.default(newmethod, tradmethod, conf.int = TRUE): x has
## more observations than y, returning perm.test(y, x, ...)
```

```
##
##  2-sample Permutation Test
##
## data:  y and x
## T = 230, p-value = 0.08571
## alternative hypothesis: true mu is not equal to 0
## 95 percent confidence interval:
##  -20  -2
```

What do you notice about the confidence interval generated?

If you run the perm.test commands, you may be seeing warnings - the function is set up so that the first sample should be larger than the second. It simply reverses the order and fixes the issue itself. If you don't like those warnings, you can turn them off. Just be careful and reading such messages so you know what order of subtraction is actually occurring.

If the observations are in a data set with a grouping variable, not stored as separate vectors, you can use a formula instead of the first 2 entries (traditional Y ~ X framework, where Y is the quantitative variable and X is a variable denoting the groups).

Data Sources

Test prep scores data were simulated.

Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to Nonparametric Statistical Methods (3rd Edition) by Hollander, Wolfe, and Chicken. Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

License